



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ИТОГОВЫЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного
обеспечения»

Создание автоматизированной системы
«Telegram-бот информационной поддержки Творческо-организационного
подразделения (ТОП) ЦКТ РТУ МИРЭА»

Студенты группы *ИКБО-43-23, Буркеев Р.Р., Данюков
К.А., Жаворонкова А.А., Савельев М.С.*

(подпись)

Преподаватель *Ильичев Г.П.*

(подпись)

Отчет
представлен *«26» сентября 2025г.*

Москва, 2025

СОДЕРЖАНИЕ

ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	7
1.1. Введение.....	7
1.1.1. Наименование программы.....	7
1.1.2. Краткая характеристика области применения.....	7
1.2. Основание для разработки, нормативные и исходные документы.....	7
1.2.1. Основание для проведения разработки.....	7
1.2.2. Наименование и условное обозначение темы разработки.....	7
1.3. Назначение разработки.....	7
1.3.1. Функциональное назначение.....	7
1.3.2. Эксплуатационное назначение.....	8
1.3.3. Задачи, решаемые в ходе разработки.....	8
1.4. Требования к программе.....	8
1.4.1. Требования к функциональным характеристикам.....	8
1.4.1.1 Требования к составу выполняемых функций.....	8
1.4.1.2 Требования к организации входных данных.....	9
1.4.1.3 Требования к организации выходных данных.....	9
1.4.2 Требования к надежности.....	9
1.4.2.1 Требования к обеспечению надежного (устойчивого) функционирования программы.....	9
1.4.2.2 Время восстановления после отказа.....	9
1.4.2.3 Отказы из-за некорректных действий пользователя.....	10
1.4.3 Условия эксплуатации.....	10
1.4.3.1 Требования к видам обслуживания.....	10
1.4.3.2 Требования к численности и квалификации персонала.....	10
1.4.4 Требования к составу и параметрам технических средств.....	10
1.4.5 Требования к информационной и программной совместимости.....	11
1.4.5.1 Требования к информационным структурам и методам решения.....	11
1.4.5.2 Требования к исходным кодам и языкам программирования.....	11
1.4.5.3 Требования к программным средствам, используемым программой.....	11
1.5. Требования к интерфейсу.....	11
1.5.1 Общие требования к интерфейсу.....	11
1.5.2 Требования к главному меню.....	11
1.5.2.1 Структура главного меню.....	12
1.5.3 Требования к меню отделов.....	12
1.5.3.1 Структура меню отделов.....	12
1.5.4 Требования к форматированию сообщений.....	12

1.6. Техничко-экономическис покатели.....	12
1.6.1 Ожидаемый экономический эффект.....	12
1.6.2 Затраты на разработку.....	12
1.7. Требования к программной документации.....	13
1.8. Порядок контроля и приемки.....	13
1.8.1 Организация приемки.....	13
1.8.2 Методы контроля качества.....	13
1.8.3 Тестовые сценарии.....	13
1.8.4 Критерии приемки.....	15
1.8.5 Документация приемки.....	15
1.8.6 Завершение приемки.....	16
1.9. Стадии и этапы разработки.....	16
1.9.1 План-график разработки.....	16
1.9.2 Форматы представления документации.....	16
ГЛАВА 2. ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ.....	17
2.1. Архитектура.....	17
2.2. Основные функции.....	17
ГЛАВА 3. ТЕСТОВАЯ ДОКУМЕНТАЦИЯ.....	18
3.1. Юз-кейсы.....	18
ГЛАВА 4. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	21
4.1 Введение.....	21
4.2 Начало работы.....	21
4.3 Основное меню.....	21
4.4 Работа с меню отделов.....	22
4.5 Решение возможных проблем.....	22
ГЛАВА 5. СИСТЕМНАЯ ДОКУМЕНТАЦИЯ.....	23
5.1 Что вам понадобится.....	23
5.2 Организация файлов.....	23
5.3 Работа в командной строке.....	23
5.4 Взаимодействие с ботом.....	24
ГЛАВА 6. ОПИСАНИЕ ОШИБОК.....	25
6.1 Неточность в приветствии нового пользователя.....	25
6.2 Неверный порядок клавиш интерфейса.....	25
6.3 Неправильная работа раздела «Словарик».....	25
6.4 Пропажа разметки markdown в разделе «Заповеди».....	26
6.5 Нерабочая опция «Назад» в разделе «Отделы».....	26
ГЛАВА 7. ДОКУМЕНТАЦИЯ ДРУГОЙ КОМАНДЫ.....	27

7.1. Техническое задание.....	27
7.2.1. Общие сведения.....	27
7.2.2. Установка и развертывание.....	28
7.2.3. Установка пакета игры для среды Godot Engine.....	28
ГЛАВА 8. ТЕСТИРОВАНИЕ ДРУГОЙ КОМАНДЫ.....	31
8.1. TC-ENV-001. Ограждение арены — невозможность покинуть границы.....	31
8.2. TC-ENV-002. Блокировка кнопки «Начать волну» при наличии противников.....	32
8.3. TC-ENV-003. Проверка изменения модели персонажа при изменении количества жизней.....	33
8.4. TC-ENV-004. Проверка обнуления скорости и прекращения движения персонажа после смерти.....	35
8.5. TC-ENV – 005. Анимация оружия.....	36
8.6. TC-ENV-006. Отсутствие элементов связанных с перезарядкой.....	37
8.7. TC-ENV-007. Враги выходят через забор за пределы огороженной арены..	38
8.8. TC-ENV-008. Враги не должны появляться за пределами арены.....	39
8.9. TC-ENV-009. Подсветка мест спавна врагов за 2–3 секунды до появления.	40
ГЛАВА 9. АНАЛИЗ ДОКУМЕНТАЦИИ ДРУГОЙ КОМАНДЫ.....	42
9.1. Общие замечания.....	42
9.1.1 Отсутствие технической документации.....	42
9.1.2 Двусмысленность формулировок и общие слова.....	42
9.1.3. Недостаточность детализации по тестам.....	43
9.1.4. Отсутствие связи между компонентами.....	43
9.1.5. Игнорирование пользовательских сценариев.....	43
9.2 Конкретные замечания.....	44
9.2.1. Неполные или отсутствующие иллюстрации.....	44
9.2.2. Повтор информации и отсутствие структуры.....	44
9.2.3. Неуказанные версии API и зависимостей.....	45
9.3 Подведение итогов.....	45
9.3.1 Общая оценка.....	45
ГЛАВА 10. ЗАКЛЮЧЕНИЕ.....	46

ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1.1. Введение

1.1.1. Наименование программы

Наименование - iBot.

Telegram-бот для Информационной поддержки
Творческо-организационного подразделения (ТОП) ЦКТ РТУ МИРЭА.

1.1.2. Краткая характеристика области применения

Программа предназначена к применению в информационно-справочном обслуживании членов ТОПа и Центра культуры и творчества.

Бот предназначен для оперативного предоставления актуальной информации о структуре, деятельности, руководстве и правилах ТОП.

1.2. Основание для разработки, нормативные и исходные документы

1.2.1. Основание для проведения разработки

Основанием для разработки является внутренняя потребность Творческо-организационного подразделения ЦКТ РТУ МИРЭА в автоматизации процесса предоставления справочной информации.

1.2.2. Наименование и условное обозначение темы разработки

Наименование темы разработки - Информационная поддержка
Творческо-организационного подразделения (ТОП) ЦКТ РТУ МИРЭА.

Условное обозначение темы разработки (шифр темы) - “ТОП-Бот-001”

1.3. Назначение разработки

1.3.1. Функциональное назначение

Функциональным назначением программы является автоматизация
процесса информационно-справочного обслуживания членов

Творческо-организационного подразделения (ТОП) и Центра культуры и Творчества путём предоставления структурированных данных о деятельности, структуре, руководстве и правилах ТОП через мессенджер Telegram.

1.3.2. Эксплуатационное назначение

Программа должна эксплуатироваться в Творческо-организационном подразделении Центра культуры и творчества (ЦКТ) РТУ МИРЭА. Конечными пользователями программы должны являться члены ТОП, студенты и сотрудники университета, нуждающиеся в получении актуальной справочной информации о подразделении.

1.3.3. Задачи, решаемые в ходе разработки

- Структурирование и централизация справочной информации (контакты, цели, правила, структура).
- Сокращение времени на поиск информации о руководителях коллективов.
- Обеспечение легкого и интуитивно понятного доступа к информации через популярный мессенджер Telegram.

1.4. Требования к программе

1.4.1. Требования к функциональным характеристикам

1.4.1.1 Требования к составу выполняемых функций

Программа должна обеспечивать возможность выполнения перечисленных ниже функций:

- а) Функция запуска диалога с пользователем (команда /start).
- б) Функция предоставления текстовой и графической информации о миссии и целях ТОП (раздел «Кто мы»).
- в) Функция отображения внутренних правил ТОП (раздел «Заповеди»).
- г) Функция предоставления информации об отделах ТОП (руководитель, контакты, задачи).

- д) Функция отображения корпоративного словаря терминов.
- е) Функция обработки ошибок (уведомление пользователя о проблемах, логирование).

1.4.1.2 Требования к организации входных данных

Входные данные программы должны быть организованы в виде:

- Текстовых файлов (в формате .ру) с данными для разделов.
- Структурированных данных (словари Python) для хранения информации об отделах.

1.4.1.3 Требования к организации выходных данных

Выходные данные программы должны быть представлены в виде сообщений, отправляемых через API Telegram, включая:

- Текстовые сообщения с форматированием Markdown.
- Сообщения об ошибках.

1.4.2 Требования к надежности

1.4.2.1 Требования к обеспечению надежного (устойчивого) функционирования программы

Надежное функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий:

- а) Организацией бесперебойного питания технических средств.
- б) Использованием лицензионного программного обеспечения.
- в) Регулярным выполнением требований по защите информации и испытаний на наличие вредоносного программного обеспечения.

1.4.2.2 Время восстановления после отказа

Время восстановления после отказа, вызванного сбоем электропитания или иными внешними факторами, не должно превышать 10 минут при условии соблюдения условий эксплуатации.

Время восстановления после отказа, вызванного неисправностью технических средств или крахом операционной системы, не должно превышать времени, требуемого на устранение неисправностей и переустановку программных средств.

1.4.2.3 Отказы из-за некорректных действий пользователя

Отказы программы возможны вследствие некорректных действий пользователя. Во избежание этого необходимо обеспечить работу программы с корректными входными данными и реализовать механизмы обработки исключений.

1.4.3 Условия эксплуатации

1.4.3.1 Требования к видам обслуживания

Программа не требует проведения специальных видов обслуживания, за исключением обновления контента и мониторинга логических ошибок.

1.4.3.2 Требования к численности и квалификации персонала

Для работы программы требуется:

- Системный администратор (обеспечивает работоспособность сервера, установку ОС и зависимостей).
- Конечный пользователь (взаимодействует с программой через Telegram-клиент).

Системный администратор должен иметь навыки администрирования ОС Linux/Windows и установки Python-окружения. Конечный пользователь должен обладать базовыми навыками работы с мессенджером Telegram.

1.4.4 Требования к составу и параметрам технических средств

В состав технических средств должен входить сервер (виртуальный или физический), включающий:

- Процессор с тактовой частотой не менее 1 ГГц.

- Оперативную память объемом не менее 512 МБ.
- Постоянную память объемом не менее 5 ГБ.
- Доступ к сети Интернет.

1.4.5 Требования к информационной и программной совместимости

1.4.5.1 Требования к информационным структурам и методам решения

Программа должна использовать структуры данных Python (словари, списки) для хранения информации. Данные должны быть организованы в соответствии с логикой работы бота.

1.4.5.2 Требования к исходным кодам и языкам программирования

Исходный код программы должен быть реализован на языке Python версии 3.10 и выше. Для взаимодействия с Telegram должен использоваться пакет `python-telegram-bot`.

1.4.5.3 Требования к программным средствам, используемым программой

Программа должна работать под управлением ОС Linux/Windows с установленным интерпретатором Python и необходимыми зависимостями.

1.5. Требования к интерфейсу

1.5.1 Общие требования к интерфейсу

Интерфейс пользователя реализуется исключительно средствами мессенджера Telegram через систему меню, кнопок и текстовых команд.

1.5.2 Требования к главному меню

После команды `/start` пользователю должно быть отправлено текстовое приветствие и отображена клавиатура с основными разделами (`ReplyKeyboardMarkup`).

1.5.2.1 Структура главного меню

Клавиатура главного меню должна иметь вид:

['Кто мы' 'Заповеди']

['Отделы' 'Словарик']

1.5.3 Требования к меню отделов

При переходе в раздел «Отделы» пользователю должна быть отображена новая клавиатура для выбора конкретного отдела.

1.5.3.1 Структура меню отделов

Клавиатура меню отделов должна иметь вид:

['Креатив' 'Документы']

['Назад']

['Перезапустить']

1.5.4 Требования к форматированию сообщений

Все текстовые сообщения должны использовать форматирование Markdown для выделения заголовков и ключевых элементов (жирный шрифт).

1.6. Технико-экономические показатели

1.6.1 Ожидаемый экономический эффект

Ожидаемый экономический эффект от внедрения системы заключается в сокращении временных затрат руководства ТОП на рутинное информирование участников подразделения не менее чем на 70%.

1.6.2 Затраты на разработку

Разработка осуществляется силами сотрудников/студентов университета без привлечения коммерческих подрядчиков, что минимизирует затраты.

1.7. Требования к программной документации

Состав программной документации должен включать в себя:

- а) техническое задание;
- б) пользовательская документация;
- в) техническая документация;
- г) текстовая документация;
- д) системная документация.

1.8. Порядок контроля и приемки

1.8.1 Организация приемки

Приемка готового продукта осуществляется заказчиком.

1.8.2 Методы контроля качества

Контроль качества осуществляется методом чёрного ящика.

1.8.3 Тестовые сценарии

Для приемки Исполнитель предоставляет Заказчику выполненные тестовые сценарии, включающий не менее 5 тест-кейсов.

UC-01 Старт

Пользователь отправляет /start. Бот просит ввести имя. После ввода имени появляется главное меню (Рисунок 1).

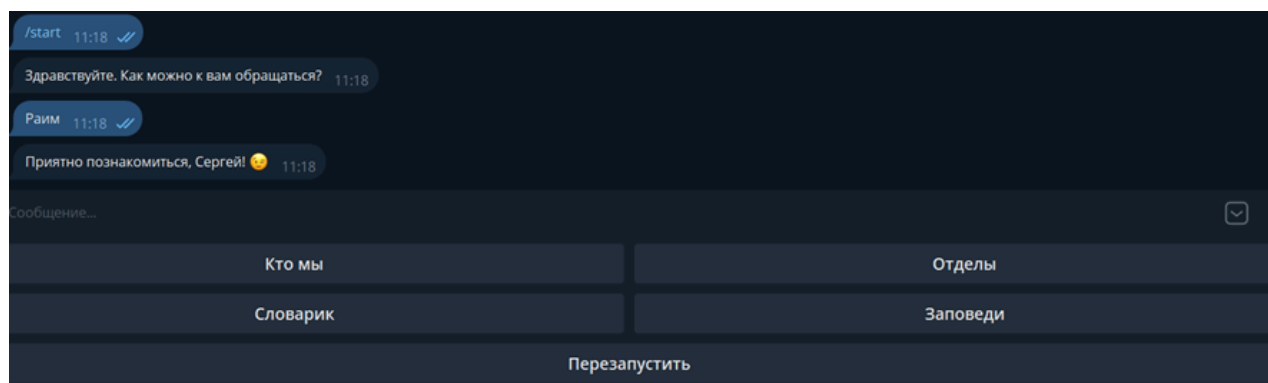


Рисунок 1 – Приветствие и главное меню

UC-02. «Кто мы»

Пользователь выбирает «Кто мы». Бот отправляет текст миссии и целей. Затем снова показывает главное меню (Рисунок 2).

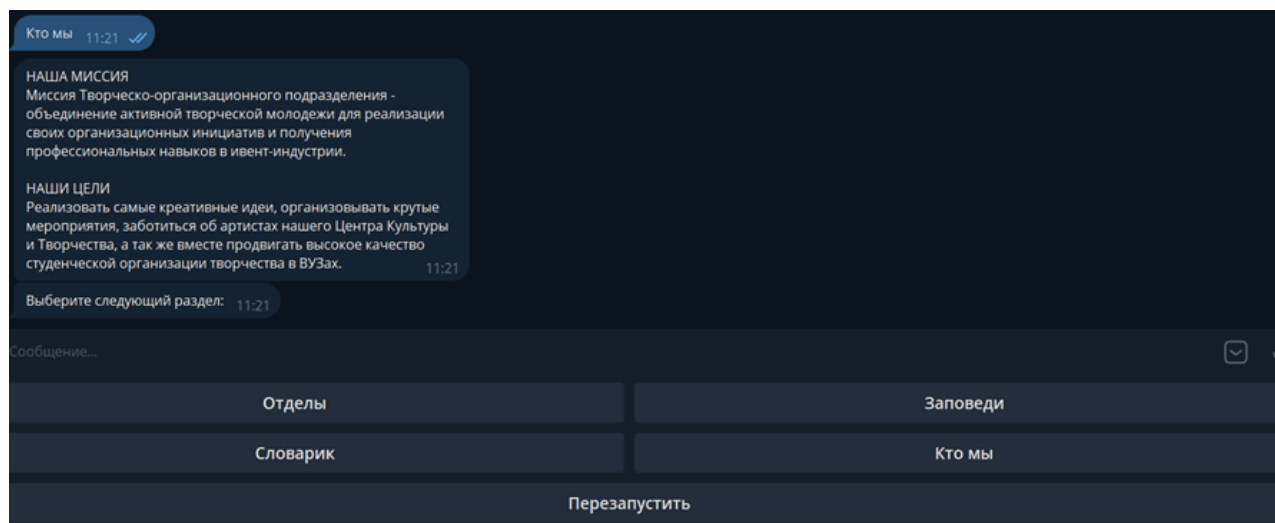


Рисунок 2 – Ознакомление по запросу

УС-03. «Заповеди»

Пользователь выбирает «Заповеди». Бот отправляет правила поведения. Затем снова главное меню (Рисунок 3).

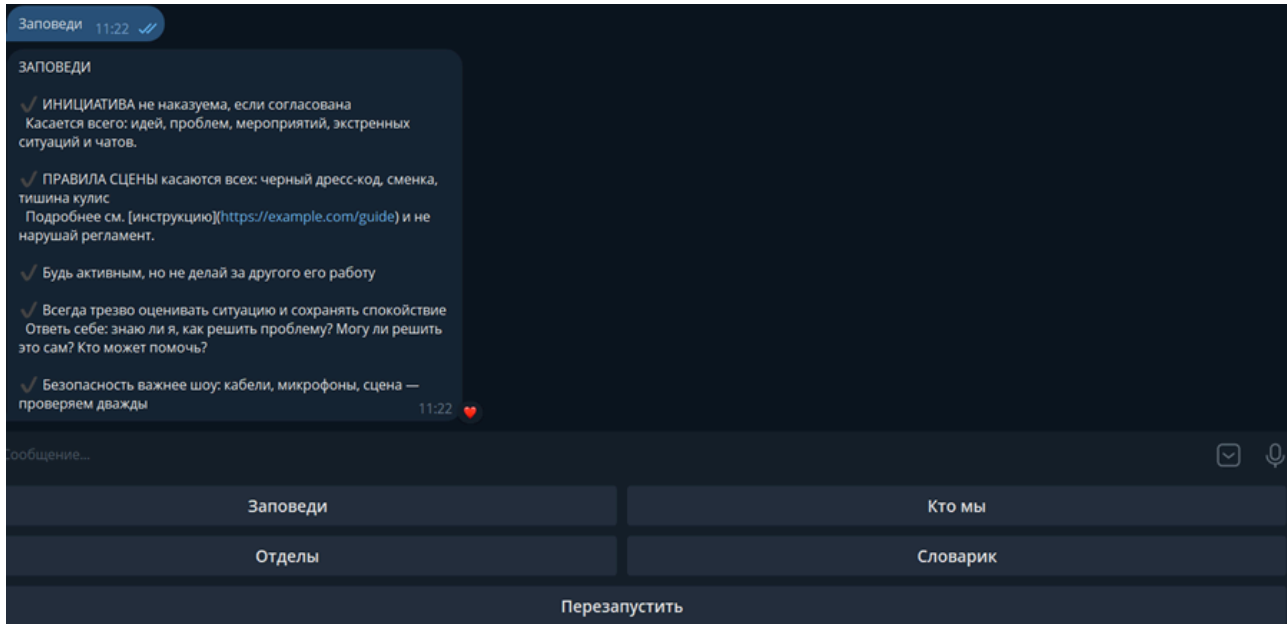


Рисунок 3 – Отображение правил по запросу

УС-04. Просмотр отдела

Пользователь выбирает «Отделы». Бот показывает клавиатуру с отделами. При выборе «Креатив» или «Документы» — отправляет описание отдела (Рисунок 4).

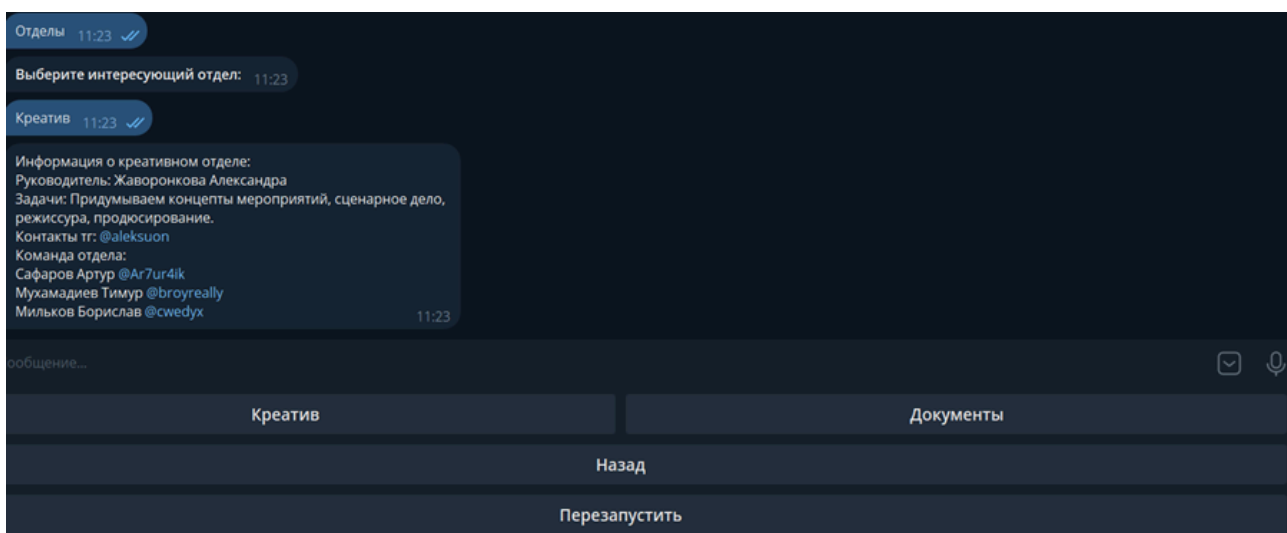


Рисунок 4 – Отображение выбранного отдела по запросу

UC-05. Перезапуск

Пользователь нажимает «Перезапустить». Бот возвращает к главному меню (Рисунок 5).

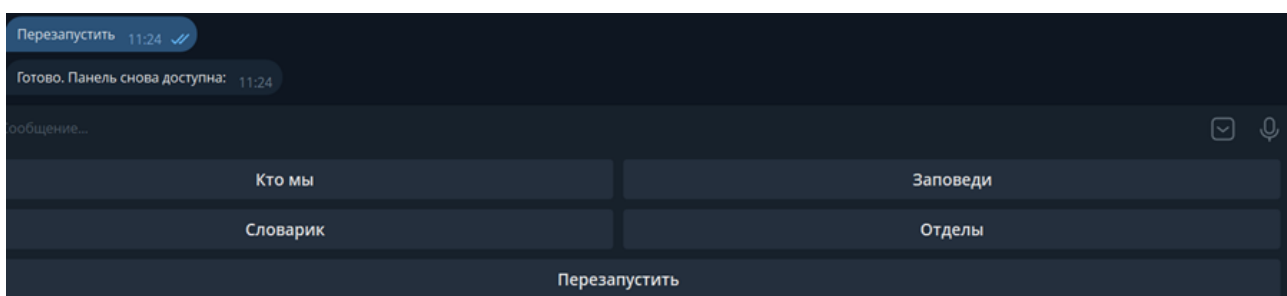


Рисунок 5 – Перезапуск и возвращение в первоначальный вид главного меню

1.8.4 Критерии приемки

Критерием успешной приемки является успешное прохождение не менее 95% всех тест-кейсов.

1.8.5 Документация приемки

После успешного прохождения испытаний подписывается Акт о приемке программного обеспечения в опытную эксплуатацию.

1.8.6 Завершение приемки

По итогам успешной опытной эксплуатации в течение установленного срока подписывается Акт о приемке программного обеспечения в промышленную эксплуатацию.

1.9. Стадии и этапы разработки

1.9.1 План-график разработки

Разработка программного обеспечения должна быть выполнена в следующие сроки (Таблица 1).

Таблица 1. План-график разработки

№	Наименование этапа	Срок исполнения	Исполнитель
1	Анализ требований и проектирование архитектуры	5 рабочих дней	Разработчик
2	Написание кода и модульное тестирование	10 рабочих дней	Разработчик
3	Наполнение контентом	3 рабочих дня	Заказчик
4	Пилотная эксплуатация и приемочные испытания	5 рабочих дней	Совместно
5	Ввод в промышленную эксплуатацию	1 рабочий день	Разработчик

1.9.2 Форматы представления документации

Вся документация должна быть предоставлена в электронном виде в форматах PDF (для руководств) и .ру (для исходного кода).

ГЛАВА 2. ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

2.1. Архитектура

Бот построен на библиотеке `python-telegram-bot` и работает как конечный автомат (FSM) с двумя состояниями:

- `MAIN_MENU` — главное меню
- `DEPARTMENTS` — меню отделов

Поток работы:

1. Пользователь вводит `/start`.
2. Бот спрашивает имя и переходит в `MAIN_MENU`.
3. В главном меню доступны кнопки: «Кто мы», «Заповеди», «Отделы», «Словарь», «Перезапустить».
4. Выбор «Отделы» переводит в состояние `DEPARTMENTS`.
5. «Перезапустить» возвращает к главному меню, `/cancel` завершает диалог.

Данные (миссия, заповеди, словарь, описания отделов) хранятся в словаре `TEXTS`.

Клавиатуры формируются через `ReplyKeyboardMarkup`.

2.2. Основные функции

- `_build_shuffled_main_keyboard()` — Собирает кнопки главного меню.
- `start()` — Приветствие, запрос имени, переход в `MAIN_MENU`.
- `main_menu()` — Обрабатывает выбор пользователя в главном меню: отправляет тексты или открывает раздел «Отделы».
- `department_menu()` — Показывает список отделов и их описание.
- `restart_handler()` — Возвращает к главному меню без повторного вопроса об имени.
- `cancel()` — Завершает работу и удаляет клавиатуру.
- `main()` — Инициализирует приложение, настраивает обработчики и запускает `polling`.

ГЛАВА 3. ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

3.1. Юз-кейсы

UC-01. Старт

Пользователь отправляет /start. Бот просит ввести имя. После ввода имени появляется главное меню (Рисунок 3.1).

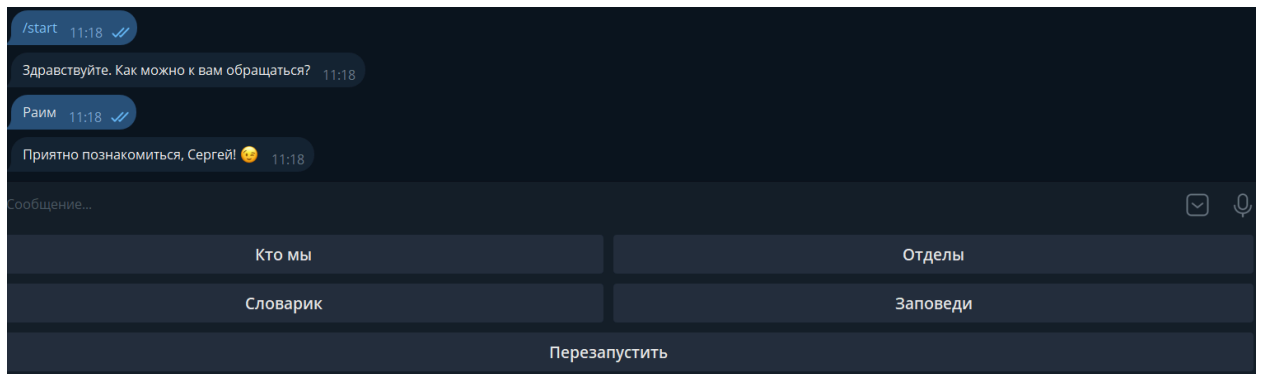


Рисунок 3.1 – Приветствие и главное меню

UC-02. «Кто мы»

Пользователь выбирает «Кто мы». Бот отправляет текст миссии и целей. Затем снова показывает главное меню (Рисунок 3.2).

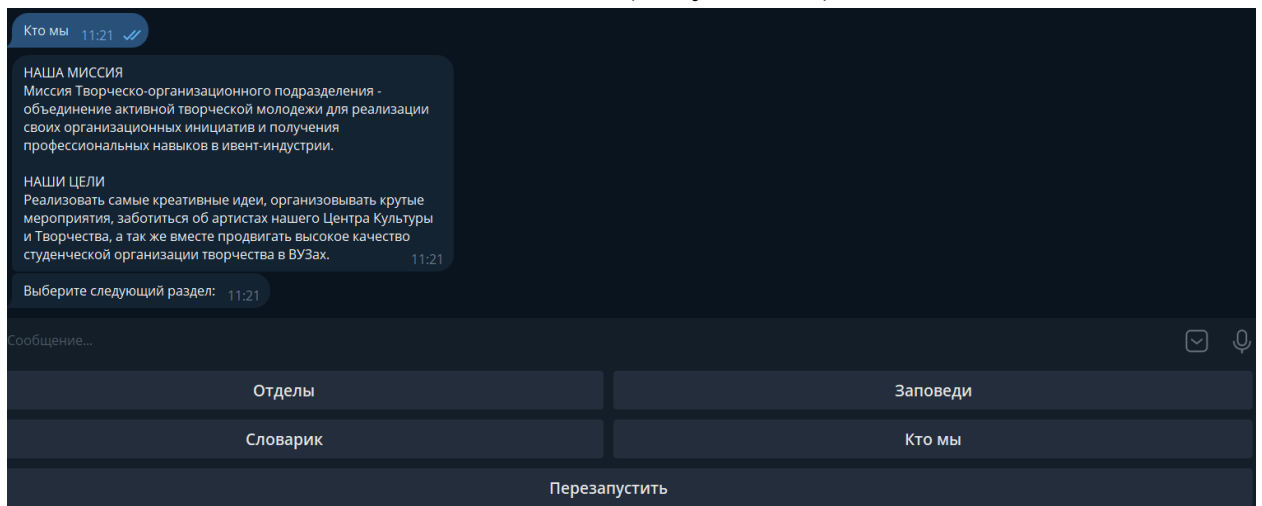


Рисунок 3.2 – Ознакомление по запросу

UC-03. «Заповеди»

Пользователь выбирает «Заповеди». Бот отправляет правила поведения. Затем снова главное меню (Рисунок 3.3).

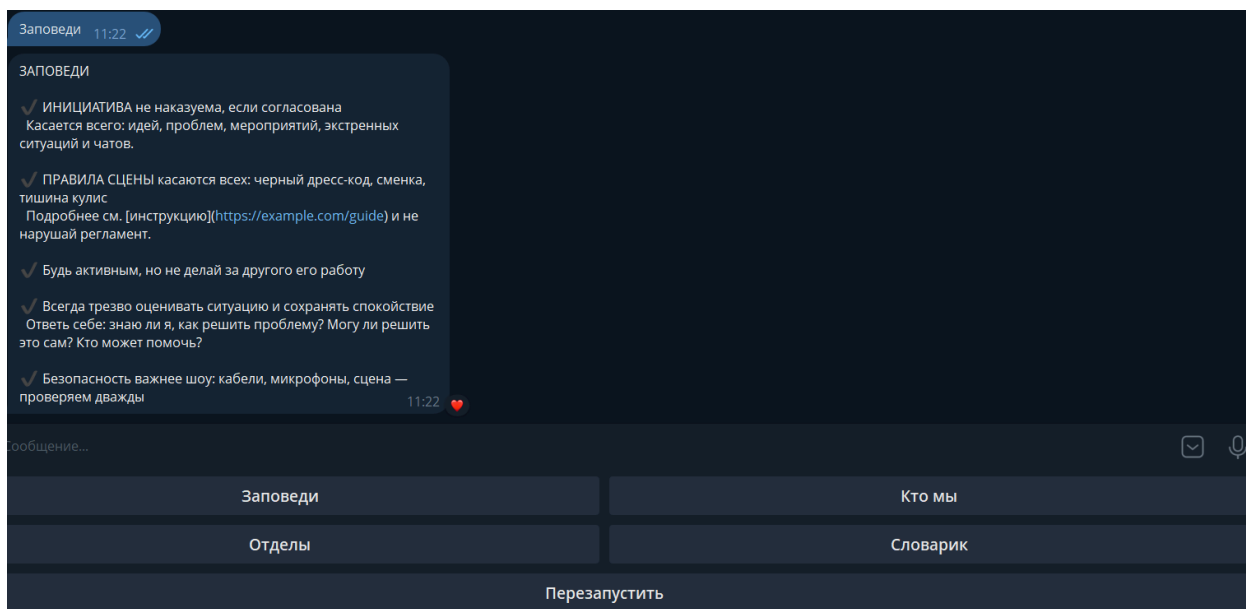


Рисунок 3.3 – Отображение правил по запросу

УС-04. Просмотр отдела

Пользователь выбирает «Отделы». Бот показывает клавиатуру с отделами. При выборе «Креатив» или «Документы» — отправляет описание отдела (Рисунок 3.4).

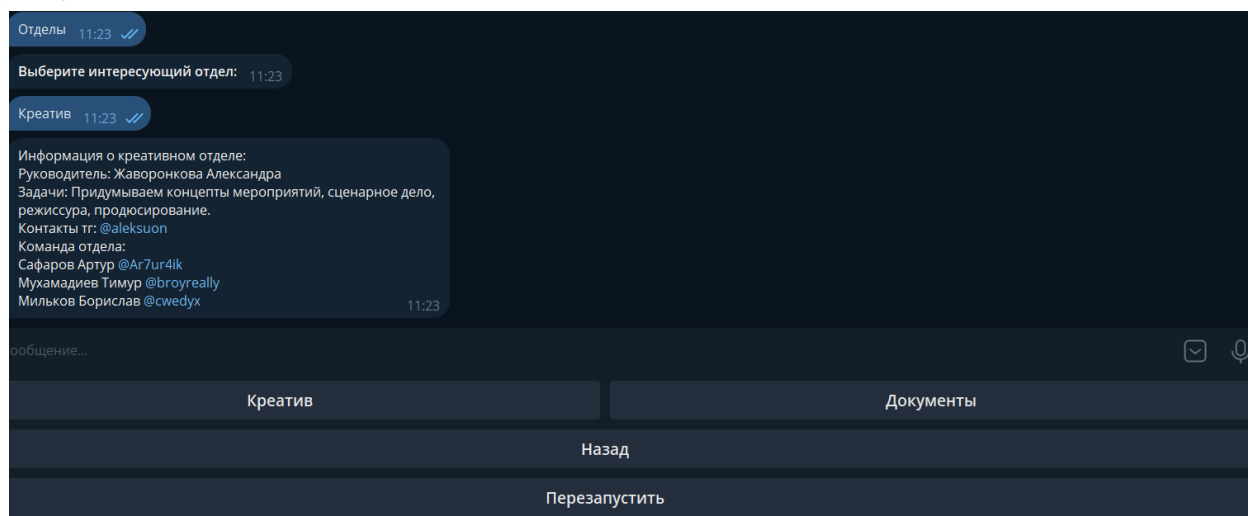


Рисунок 3.4 – Отображение выбранного отдела по запросу

УС-05. Перезапуск

Пользователь нажимает «Перезапустить». Бот возвращает к главному меню (Рисунок 3.5).

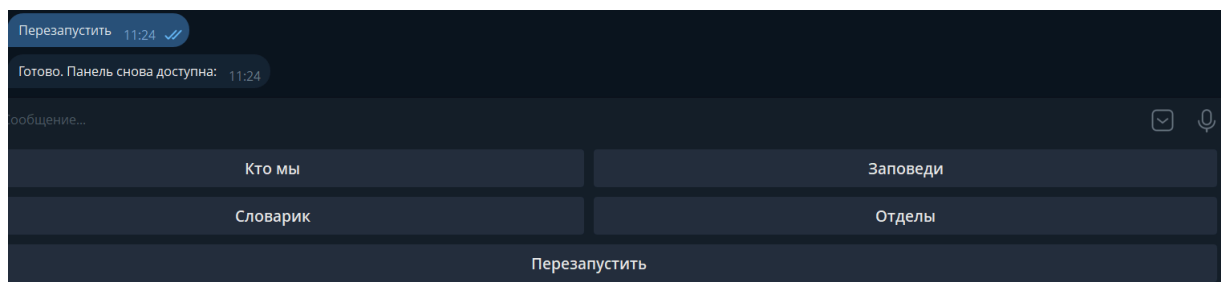


Рисунок 3.5 – Перезапуск и возвращение в первоначальный вид главного меню

ГЛАВА 4. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1 Введение

Настоящее руководство предназначено для конечных пользователей телеграм-ботам iBot. Бот предоставляет легкий доступ к ключевой информации о описанном подразделении: его миссии, правилах внутренней работы, структуре отделов и корпоративной терминологии.

4.2 Начало работы

Для начала взаимодействия с ботом выполните следующие шаги:

- Инициализация. Нажмите кнопку START или введите команду /start в поле для сообщений.
- Знакомство. Бот запросит ваше имя. Введите любое удобное для обращения имя и отправьте сообщение.

4.3 Основное меню

После знакомства откроется главное меню, представленное панелью с четырьмя кнопками.

- Кнопка «Кто мы» – предоставляет текст с описанием миссии и стратегических целей подразделения.
- Кнопка «Заповеди» – отображает свод основных правил и принципов работы. В некоторых случаях форматирование текста (жирный шрифт, курсив) может отсутствовать.
- Кнопка «Отделы» – открывает меню выбора отделов для получения детальной информации.
- Кнопка «Словарик» – выводит обширный корпоративный словарь с расшифровкой специфических терминов, аббревиатур и названий локаций.
- Для навигации также всегда доступна кнопка «Перезапустить», которая возвращает пользователя в главное меню.

4.4 Работа с меню отделов

- В главном меню нажмите кнопку «Отделы».
- На экране появится новая панель с кнопками: «Креатив», «Документы» и «Назад».
- Для получения информации о конкретном отделе нажмите на соответствующую кнопку. Бот пришлет сообщение с данными о руководителе, задачах отдела и контактах.
- Для возврата к списку отделов используйте кнопку «Назад».

4.5 Решение возможных проблем

- Сброс текущего сеанса. Если бот перестал отвечать или вы заблудились в меню, введите команду /restart. Это вернет вас в главное меню без повторного запроса имени.
- Полный перезапуск. Чтобы начать взаимодействие с бота заново (включая этап знакомства), используйте команду /start.
- Завершение работы. Для завершения работы с ботом введите команду /cancel.

ГЛАВА 5. СИСТЕМНАЯ ДОКУМЕНТАЦИЯ

5.1 Что вам понадобится

Компьютер или сервер с выходом в интернет

Установленный Python (версия 3.7 или новее)

Доступ к приложению Telegram

5.2 Организация файлов

Скачайте bot.py

Переместите его в папку на любом доступном диске (C, D и т.д.)

5.3 Работа в командной строке

Введем условные обозначения:

foldername - название папки куда вы закинули бота

Запустите Powershell

Далее выполняйте команды по порядку:

```
cd foldername
```

Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass (это одна длинная команда)

```
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

```
pip install --upgrade pip
```

```
pip install python-telegram-bot==20.3
```

```
python bot.py
```

Закрытие окна Powershell приведет к окончанию сессии.

Чтобы вновь ее запустить, достаточно будет ввести две команды:

```
cd foldername
```

```
python bot.py
```

5.4 Взаимодействие с ботом

Введите в поисковой строке в telegram:

@student_test_ugh_bot

Введите /start

Можно приступать к тестированию.

ГЛАВА 6. ОПИСАНИЕ ОШИБОК

6.1 Неточность в приветствии нового пользователя

При первой инициализации, бот интересуется тем, как ему называть новоприбывшего клиента. Пользователь вводит свое имя. Здесь и происходит неправильная работа бота: вместо того, чтобы поприветствовать гостя корректно, он называет его намеренно неправильным именем. В программном коде это реализовано так: имя пользователя сравнивается со списком имен, и выбирается случайное из списка отличных от того, что ввел клиент.

Тип ошибки: логическая.

Способ обнаружения: первичное взаимодействие с продуктом.

6.2 Неверный порядок клавиш интерфейса

После взаимодействия с любой из представленных опций и выхода в главное меню, все доступные для нажатия варианты перемешиваются. Порядок реализован в программном коде в виде псевдорандома.

Тип ошибки: логическая

Способ обнаружения: последовательные использование любой из доступных опций и выход в главное меню.

6.3 Неправильная работа раздела «Словарик»

При непосредственной работе с разделом «Словарик», не происходит ожидаемого показа внутренней информации раздела. Вместо этого, нам снова предлагают выбрать один из вариантов. Происходит так потому, что в программном коде нет никакого раздела «Словарик». Есть только «Словарь», и при поиске, программа не может выдать корректный вариант. Все, что ей остается, так это вернуться к первоначальному варианту меню.

Тип ошибки: архитектурная

Способ обнаружения: попытка перейти в раздел «Словарик».

6.4 Пропажа разметки markdown в разделе «Заповеди»

В разделе «Заповеди», присутствует много markdown-разметки в тексте. Но с шансом в 60%, при открытии раздела, markdown-разметка может пропасть. Останется лишь не отформатированный текст.

Тип ошибки: рендеринговая

Способ обнаружения: последовательные вход и выход в раздел «Заповеди», вплоть до обнаружения ошибки.

6.5 Нерабочая опция «Назад» в разделе «Отделы»

В каждом из выбранных вариантов, всегда присутствует клавиша «Назад». Она помогает вернуться и выбрать нужный пользователю раздел. Но именно в разделе «Отделы», помогает только лишь клавиша «Перезапустить».

Тип ошибки: логическая

Способ обнаружения: попытка вернуться из раздела «Отделы» с помощью клавиши «Назад»

ГЛАВА 7. ДОКУМЕНТАЦИЯ ДРУГОЙ КОМАНДЫ

7.1. Техническое задание

Техническое задание отсутствует. Не удовлетворяет условиям.

7.2. Системная документация

Системная документация по игре “VOXGORE”

7.2.1. Общие сведения

Наименование продукта: “VOXGORE”

Версия: 0.0.0.1

Дата сборки: 08.09.2025

Движок: Godot Engine 4.4.1

Тип приложения: 3D-шутер

Системные требования:

Минимальные:

- ОС: Windows 10
- Процессор: Intel Core i3
- Память: 2 GB RAM
- Видеокарта: Vulkan 1.0 совместимая
- Место на диске: 110 MB
- Поддержка Vulkan

Рекомендуемые:

- ОС: Windows 10+
- Процессор: Intel Core i5
- Память: 4 GB RAM
- Видеокарта: Vulkan 1.2 совместимая
- Место на диске: 110 MB
- Поддержка Vulkan

7.2.2. Установка и развертывание

Разархивируйте пакет voxgore_export.rar

Запустите Voxgore Bloodsuckers.exe

7.2.3. Установка пакета игры для среды Godot Engine

1. Установите последнюю версию Godot Engine (https://store.steampowered.com/app/404790/Godot_Engine/)
2. Разархивируйте VOXGORE-Developers-rep.rar и откройте project.godot в Godot Engine
3. Включите аддоны (Рисунок 7.1-4)

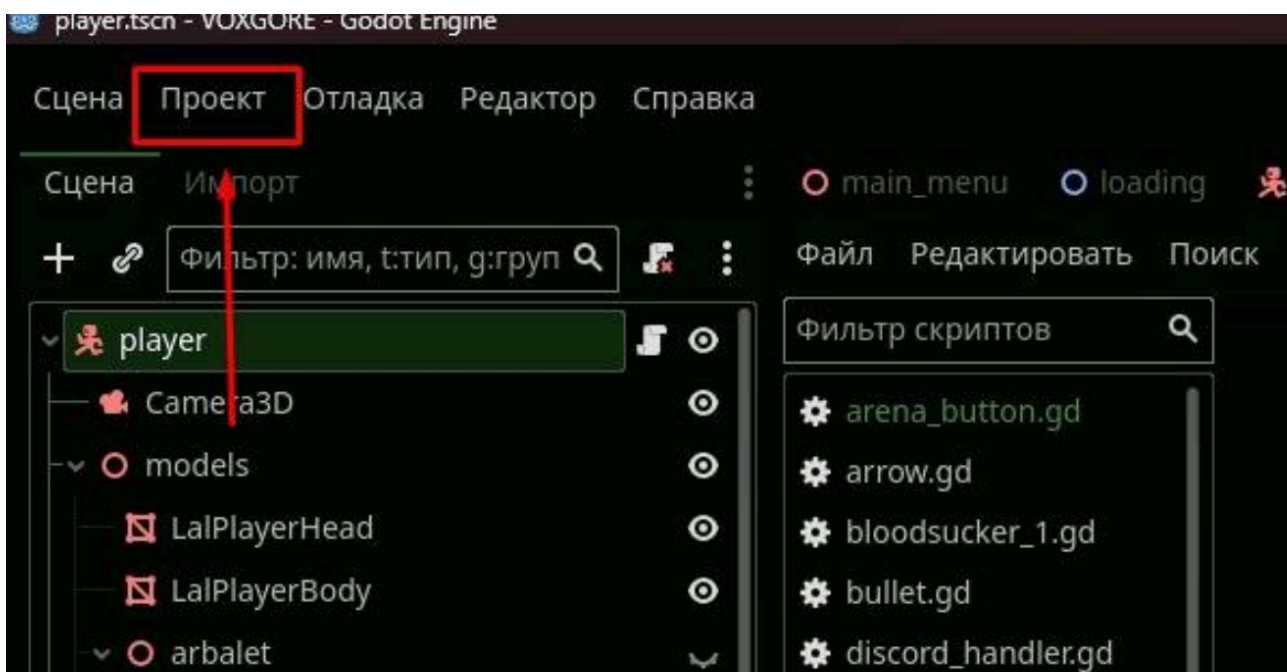


Рисунок 7.1 - Настройка Godot Engine

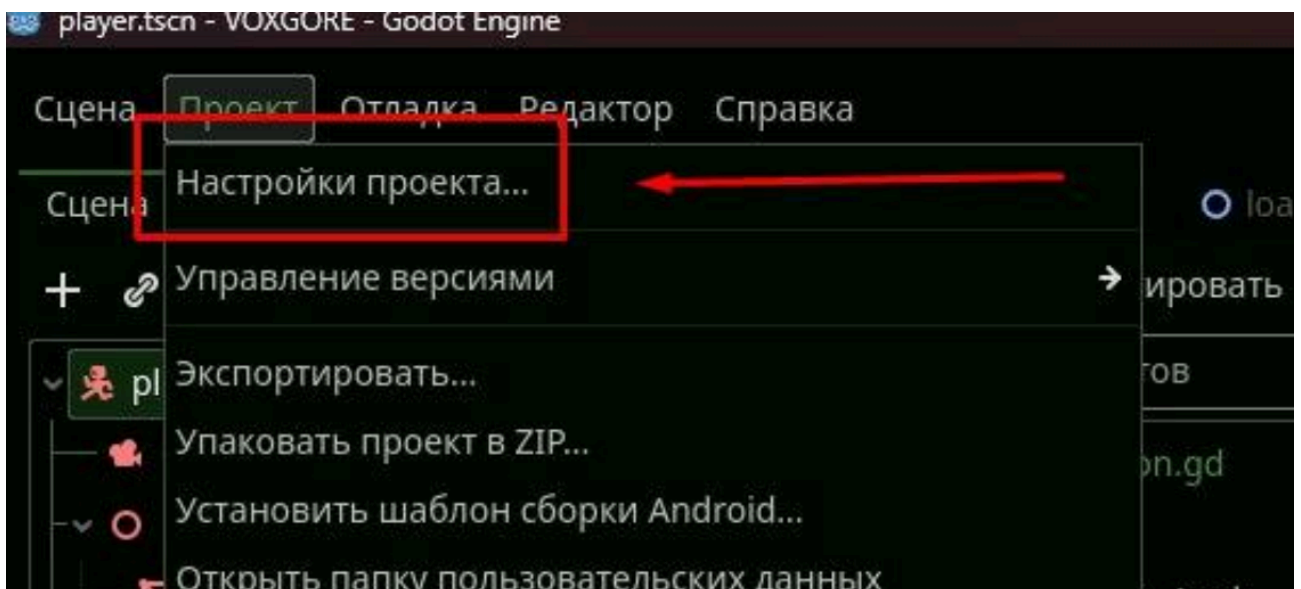


Рисунок 7.2 - Настройки проекта

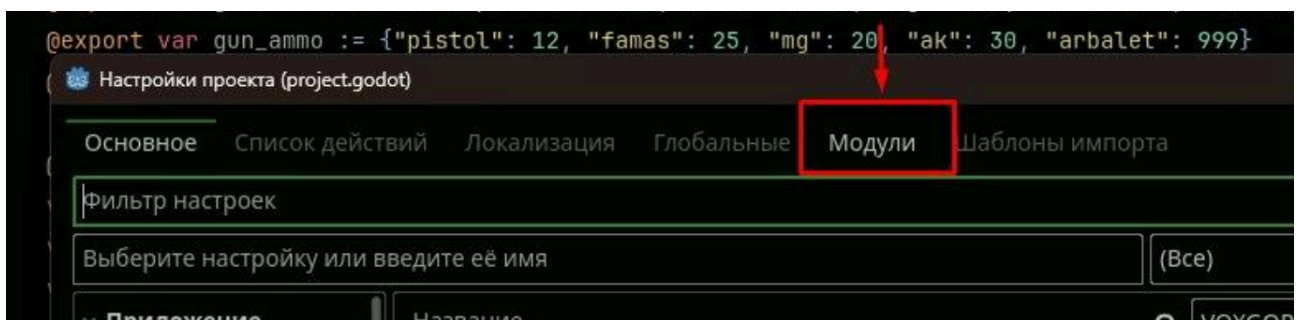


Рисунок 7.3 - Переход во вкладку модули

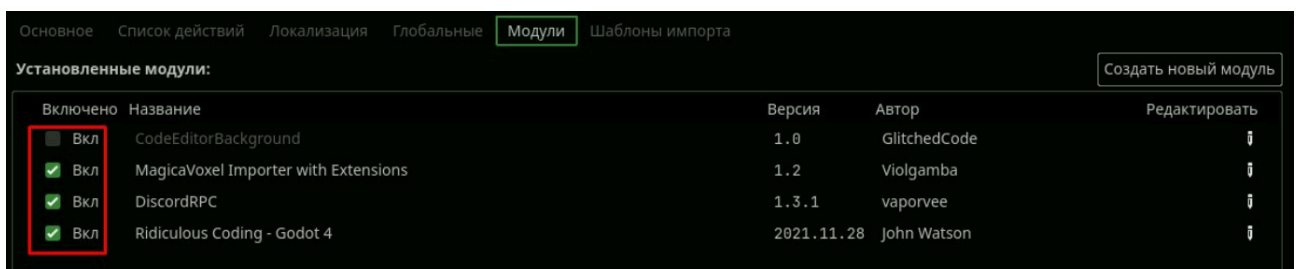


Рисунок 7.4 - Настройки установленных модулей

7.3. Тестовая документация

Тесты окружения:

1. Персонаж не должен иметь возможность выйти за пределы забора, огораживающего арену с врагами.
2. Персонаж не должен иметь возможность нажать на кнопку начала волны, пока на арене есть противники.

Тесты персонажа:

1. Моделька персонажа должна меняться при изменении количества жизней.

2. При смерти скорость персонажа должна быть равна нулю и он не должен двигаться.

Тесты оружия:

1. На каждом виде оружия должна корректно работать анимация стрельбы: при нажатии кнопки выстрела проигрывается анимация стрельбы, при отпускании кнопки выстрела анимация прекращается.

2. При перезарядке и подборе одинакового оружия таймер перезарядки должен сбрасываться.

Тесты врагов:

1. Враги не должны иметь возможность выйти за пределы забора, огораживающего арену.

2. Места появления врагов должны подсвечиваться красными кругами за пару секунд до их появления.

3. Враги не должны появляться за пределами арены

ГЛАВА 8. ТЕСТИРОВАНИЕ ДРУГОЙ КОМАНДЫ

8.1. TC-ENV-001. Ограждение арены — невозможность покинуть границы

Степень ошибки: средняя.

1) Идентификатор: TC-ENV-001

2) Название: Ограждение арены — невозможность покинуть границы

3) Описание: Проверить, что персонаж не может выйти за пределы забора, ограждающего арену с врагами.

4) Предварительные условия:

а) Скачан Godot Engine

б) Запущен Godot Engine

5) Шаги выполнения:

а) Подойти к ограждению арены по периметру, уделив внимание известным стыкам секций забора.

б) В точке узкого зазора/стыка (см. координаты/скриншот прилагаемыми материалами) попытаться пройти/выпрыгнуть/сделать перекат в сторону внешней области.

в) Повторить попытки под разными углами и с различной скоростью передвижения.

6) Ожидаемый результат: Коллизии забора и/или навмеша не позволяют покинуть пределы арены; персонаж останавливается у ограждения, остаётся в пределах сцены, игровая логика волн не нарушается.

7) Фактический результат: Персонаж может покинуть арену через узкий зазор/ошибку коллизии на стыке секций забора (воспроизводится стабильно в конкретной точке по периметру). После выхода за пределы арены логика боя нарушается (враги теряют агро/волна зависает).

8) Статус: Failed

9) Подтверждение Рисунок 8.1.



Рисунок 8.1 – Некорректный выход за пределы игровой арены

8.2. ТС-ENV-002. Блокировка кнопки «Начать волну» при наличии противников

- 1) Идентификатор: ТС-ENV-002
- 2) Название: Блокировка кнопки «Начать волну» при наличии противников
- 3) Описание: Проверить, что при наличии противников на арене невозможно нажать кнопку начала новой волны.
- 4) Предварительные условия:
 - а) Скачан Godot Engine
 - б) Запущен Godot Engine
- 5) Шаги выполнения:
 - а) Открыть интерфейс управления волнами/зайти в зону триггера начала волны.
 - б) Найти элемент управления «Начать волну» и попытаться активировать его.
 - 6) Ожидаемый результат: Кнопка «Начать волну» присутствует в UI, но недоступна (disabled), пока на арене есть противники.

7) Фактический результат: При взаимодействии с кнопкой не следует никакой реакции, так как на поле боя присутствуют враги. Все работает так, как должно.

8) Статус: Passed

9) Подтверждение Рисунок 8.2.



Рисунок 8.2 – В присутствии врагов, при взаимодействии с кнопкой, новая волна не начинается

8.3. TC-ENV-003. Проверка изменения модели персонажа при изменении количества жизней

1) Идентификатор: TC-ENV-003

2) Название: Проверка изменения модели персонажа при изменении количества жизней.

3) Описание: Убедиться, что визуальная модель персонажа изменяется соответствующим образом при получении урона или восстановлении здоровья.

4) Предварительные условия:

а) Скачан Godot Engine

б) Запущен Godot Engine

5) Шаги выполнения:

- а) запомнить исходный вид модели персонажа;
 - б) уменьшить количество жизней персонажа (например, получить урон от противника или среды);
 - в) пронаблюдать за изменением модели персонажа;
 - г) восстановить количество жизней персонажа (если применимо);
 - д) пронаблюдать за изменением модели персонажа.
- 6) Ожидаемый результат: Внешний вид модели персонажа изменяется (например, появляются визуальные повреждения, кровотечение, меняется цветовая гамма) при уменьшении количества жизней. При восстановлении здоровья модель возвращается к исходному или менее поврежденному виду.
- 7) Фактический результат: Модель персонажа корректно меняется при получении урона: появились визуальные повреждения и окровавленные текстуры. После восстановления здоровья модель вернулась к исходному состоянию. Ошибок не выявлено.
- 8) Статус: Passed
- 9) Подтверждение Рисунок 8.3 и 8.4.



Рисунок 8.3 – Если здоровье полное, специфичные анимации персонажа отсутствуют

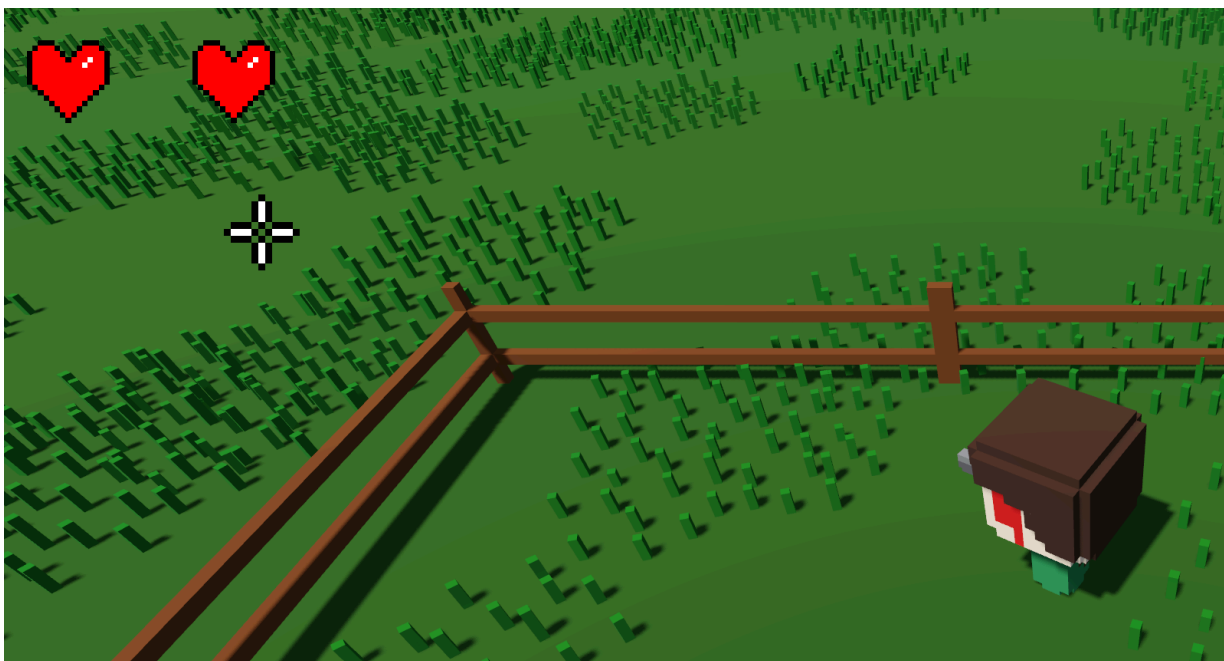


Рисунок 8.4 - Если же персонаж поврежден, на его теле выступает кровь

8.4. ТС-ENV-004. Проверка обнуления скорости и прекращения движения персонажа после смерти

- 1) Идентификатор: ТС-ENV-004
- 2) Название: Проверка обнуления скорости и прекращения движения персонажа после смерти.
- 3) Описание: Убедиться, что после смерти всякое движение персонажа прекращается, а его скорость становится равной нулю, независимо от действий до момента смерти.
- 4) Предварительные условия:
 - а) Скачан Godot Engine
 - б) Запущен Godot Engine
- 5) Шаги выполнения:
 - а) начать движение персонажем в любом направлении;
 - б) добиться смерти персонажа во время движения (получить смертельный урон от противника);
 - в) пронаблюдать за поведением персонажа после наступления смерти.
- 6) Ожидаемый результат: В момент смерти анимация движения прекращается, скорость персонажа становится равной нулю. Персонаж более не

реагирует на команды движения и остается неподвижным до момента возрождения.

7) Фактический результат: В момент смерти анимация смерти проигралась, но физическое тело персонажа продолжило движение по инерции (скольжение по поверхности). Персонаж двигался после наступления смерти, что является ошибкой.

8) Статус: Failed

9) Подтверждение невозможно.

8.5. TC-ENV – 005. Анимация оружия

Критическая ошибка: Не работает анимация стрельбы оружия.

1) Идентификатор: TC-ENV-005

2) Название: Анимация оружия

3) Описание: На каждом виде оружия должна корректно работать анимация стрельбы: при нажатии кнопки выстрела проигрывается анимация стрельбы, при отпускании кнопки выстрела анимация прекращается.

4) Предварительные условия:

а) Скачан Godot Engine

б) Запущен Godot Engine

5) Шаги воспроизведения:

а) Запустить игру.

б) Подобрать любое оружие.

в) Нажать кнопку выстрела.

г) Отпустить кнопку выстрела.

6) Ожидаемый результат: При шаге 3 проигрывается анимация стрельбы. При шаге 4 анимация прекращается.

7) Фактический результат: Анимация стрельбы не проигрывается ни на одном из шагов. Персонаж не совершает никаких действий, связанных со стрельбой.

8) Статус: Failed

9) Подтверждение Рисунок 8.5.



Рисунок 8.5 – Анимации не воспроизводятся согласно техническому заданию

8.6. ТС-ENV-006. Отсутствие элементов связанных с перезарядкой

Средняя ошибка: Отсутствует интерфейс и логика перезарядки оружия.

- 1) Идентификатор: ТС-ENV-006
- 2) Название: Отсутствие элементов связанных с перезарядкой
- 3) Описание: В интерфейсе полностью отсутствует счетчик оставшихся патронов и таймер перезарядки. Невозможно проверить сброс таймера при подборе одинакового оружия, так сама механика не реализована.

4) Предварительные условия:

- а) Скачан Godot Engine
- б) Запущен Godot Engine

5) Шаги воспроизведения:

- а) Запустить игру.
- б) Подобрать оружие.
- в) Произвести несколько выстрелов (не работает).
- г) Подобрать такое же оружие с земли.

6) Ожидаемый результат: На экране отображается счетчик патронов. После шага 4 таймер перезарядки сбрасывается, а счетчик патронов обновляется.

7) Фактический результат: Интерфейс с данными о патронах и перезарядке отсутствует. Проверить поведение системы невозможно.

8) Статус: Failed

9) Подтверждение невозможно в связи с отсутствием данных.

8.7. ТС-ENV-007. Враги выходят через забор за пределы огороженной арены

Критическая ошибка: Не работает анимация стрельбы оружия.

1) Идентификатор: ТС-ENV-007

2) Название: Выход врагов за пределы арены через ограждение

3) Описание: В ограждении игрового поля присутствует дефект — неправильно настроенная коллизия, позволяющая врагам проходить сквозь забор и покидать пределы игровой зоны. Визуально забор цельный, но коллизия не срабатывает в конкретной точке, что нарушает игровой процесс.

4) Предварительные условия:

а) Скачан Godot Engine

б) Запущен Godot Engine

5) Шаги выполнения:

а) Запустить игру.

б) Дождаться появления врагов на арене.

в) Направить врагов к ограждению.

г) Попытаться провести врагов сквозь забор в проблемной точке.

д) Повторить попытку под разными углами и скоростями.

6) Ожидаемый результат: Забор по всему периметру имеет сплошную коллизию; юниты (игрок и враги) не могут покинуть арену.

7) Фактический результат: В указанной точке юнит проходит сквозь забор и оказывается за пределами игровой арены.

8) Статус: Failed

9) Подтверждение Рисунок 8.6.

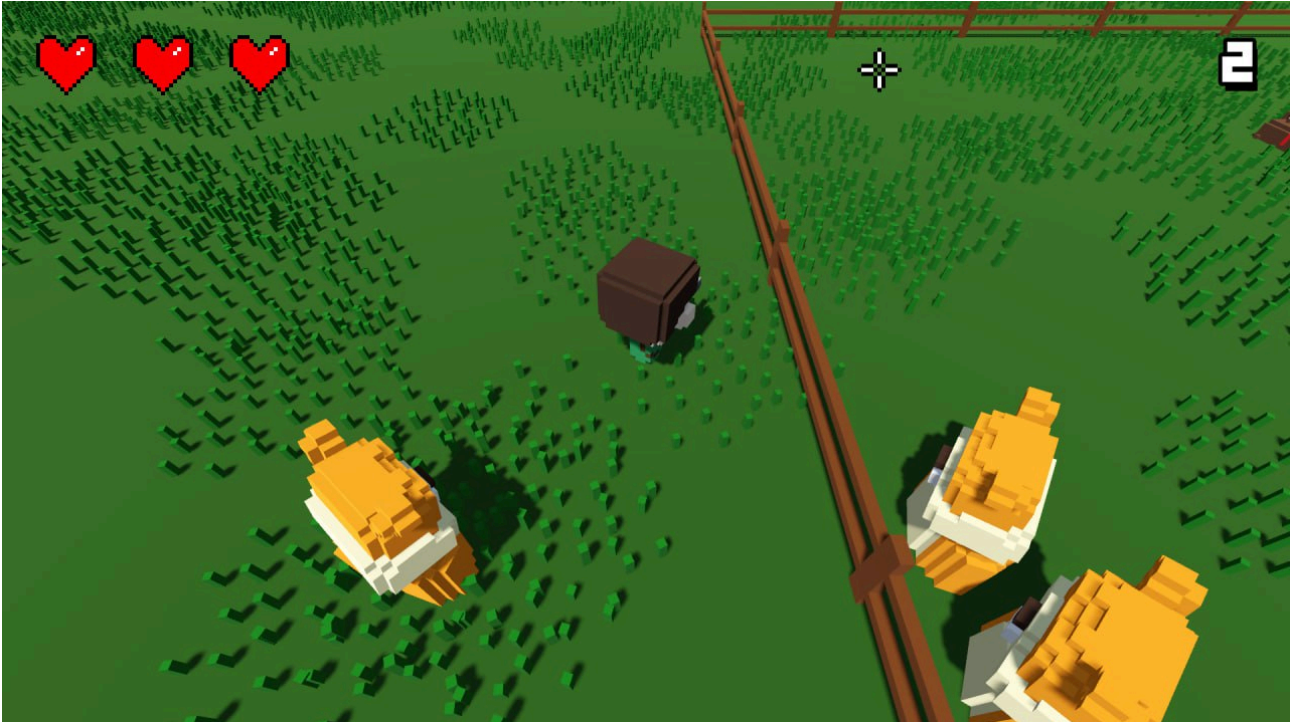


Рисунок 8.6 – Наглядная демонстрация покинувших арену игрока и юнитов

8.8. TC-ENV-008. Враги не должны появляться за пределами арены

- 1) Идентификатор: TC-ENV-008
- 2) Название. Враги не должны появляться за пределами арены.
- 3) Описание. Убедиться, что механизм спавна врагов функционирует корректно, и все враги появляются строго внутри игровой арены, ограниченной ограждением.
- 4) Предварительные условия:
 - а) Скачан Godot Engine
 - б) Запущен Godot Engine
- 5) Шаги выполнения:
 - а) Запустить игру;
 - б) Дождаться появления врагов на арене;
 - в) Осмотреть периметр арены и область за её пределами.
 - г) Зафиксировать точки появления врагов.

6) Ожидаемый результат. Все враги появляются строго в пределах игровой арены, внутри ограждения. Появление врагов за пределами арены не наблюдается.

7) Фактический результат. Все враги появляются строго в пределах игровой арены, внутри ограждения. Дефекты не обнаружены.

8) Статус. Passed.

9) Подтверждение невозможно в связи с правильностью тестирования.

8.9. TC-ENV-009. Подсветка мест спавна врагов за 2–3 секунды до появления

1) Идентификатор: TC-ENV-009

2) Название. Места появления врагов должны подсвечиваться красными кругами за пару секунд до их появления.

3) Описание. Убедиться в корректности работы системы визуального оповещения о появлении врагов. За несколько секунд до спавна противников, место их появления должно быть обозначено на игровой арене подсвечивающимся красным кругом.

4) Предварительные условия:

а) Скачан Godot Engine

б) Запущен Godot Engine

5) Шаги выполнения:

а) Запустить игру;

б) Дождаться появления врагов на арене;

в) Внимательно наблюдать за поверхностью арены до момента появления врагов.

г) Зафиксировать момент и место появления визуальной метки.

д) Зафиксировать момент появления врага и его соответствие ранее показанной метке.

6) Ожидаемый результат. За 2–3 секунды до появления каждого врага или группы врагов в точке их спавна появляется четко видимая подсветка.

7) Фактический результат. За 2–3 секунды до появления каждого врага или группы врагов в точке их спавна появляется четко видимая подсветка. Дефекты не обнаружены.

8) Статус. Passed.

9) Подтверждение Рисунок 8.7.



Рисунок 8.7 – Место прибытия монстров отмечено красным кругом

ГЛАВА 9. АНАЛИЗ ДОКУМЕНТАЦИИ ДРУГОЙ КОМАНДЫ

9.1. Общие замечания

9.1.1 Отсутствие технической документации

В документации отсутствует единый технический документ, который можно было бы назвать полноценным техническим заданием. В представленном PDF-файле упоминается структура проекта, тесты, системные требования, инструкция по запуску и краткое руководство пользователя, но при этом не описаны цели разработки, функциональные требования, сценарии использования, ожидаемое поведение системы и её архитектурная логика. Это нарушает базовые требования к документации ПО, так как не позволяет стороннему разработчику или тестировщику понять исходные проектные намерения. Рекомендуется составить техническое задание по ГОСТ 19.201.

9.1.2 Двусмысленность формулировок и общие слова

Многие описания носят декларативный характер, не фиксируя точные спецификации. Например, в разделе "Игровой процесс" говорится, что "основная задача игрока — уничтожение волн противников", но не уточняется, сколько волн предусмотрено, как нарастает сложность, какие именно механики задействованы (паттерны спавна, логика агро, ИИ и пр.). Это затрудняет формализацию требований, написание тестов и проведение аудита. В части "Архитектура" говорится, что папка `scenes` содержит "основные строительные блоки игры", однако не указано, какие конкретно сцены входят, как они взаимосвязаны и какие скрипты в них подключены. Формулировки необходимо заменить на более конкретные: например, "сцена `main_menu.tscn` содержит UI выбора режима; сцена `arena.tscn` реализует логику боя и спавна противников".

9.1.3. Недостаточность детализации по тестам

Тестовая документация имеет слабую формальную структуру. Тесты сведены в нумерованный список без подробного описания. Отсутствуют шаги воспроизведения, идентификаторы, условия выполнения и скриншоты. Например, пункт "При смерти скорость персонажа должна быть равна нулю" не содержит описания, как это проверялось, какой конкретно результат был получен, и воспроизводится ли баг. Такой формат не подходит для передачи в QA-отдел или внешнюю экспертизу. Рекомендуется переработать все тест-кейсы по формату IEEE 829 или аналогичной структуре: предусмотреть шаги, ожидания, факты и критерии успеха.

9.1.4. Отсутствие связи между компонентами

В документации отсутствуют связи между архитектурным описанием и тестовой частью. Например, в "Архитектуре" упоминаются скрипты `player.gd` и `bloodsucker.gd`, однако в тестах не уточняется, что именно проверяется из поведения этих скриптов. Неясно, какие компоненты ответственны за агро врагов, управление волнами, отображение интерфейса. Нет схемы связей между сценами, контроллерами, UI и логикой. Это создает ощущение документации "в отрыве" от кода. Рекомендуется добавить диаграмму классов или диаграмму компонентов, и в каждом разделе тестов указывать, какие элементы системы участвуют в тестируемом поведении.

9.1.5. Игнорирование пользовательских сценариев

Руководство пользователя слишком общее и не охватывает крайние случаи, типичные ошибки, нестандартные сценарии. Например, не указано, что делать, если игра не запускается, каковы минимальные действия при зависании, как происходит сохранение прогресса (если предусмотрено), какие клавиши

управления, можно ли использовать геймпад и т.д. Руководство состоит из трёх пунктов и не покрывает реальных кейсов использования. Требуется дополнение разделами "Часто задаваемые вопросы", "Управление", "Ошибки и сбои", "Минимальные навыки для пользователя".

9.2 Конкретные замечания

9.2.1. Неполные или отсутствующие иллюстрации

В документации приведены упоминания "Рисунок 1", "Рисунок 2" и т.д., но сами изображения отсутствуют или не подписаны. Это создаёт ощущение незавершённости и снижает понятность текста. В частности, в разделе настройки Godot Engine указано: "Включите аддоны (рис 1-4)", но не поясняется, что именно на этих изображениях показано, какие параметры менять и где они расположены в интерфейсе. Требуется либо встроить полноразмерные скриншоты с подписями, либо оформить их отдельным приложением с референсами из текста.

9.2.2. Повтор информации и отсутствие структуры

Некоторые сведения дублируются. Например, системные требования прописаны как в разделе системной документации, так и в пользовательском руководстве, но при этом формулировки и формат подачи различаются. В одних случаях используются маркеры, в других — списки с дефисами, без единообразия. Отсутствует оглавление и сквозная нумерация разделов, что делает навигацию по документу неудобной. Следует унифицировать стиль оформления и структурировать документ по логике: вводные данные → архитектура → установка → руководство → тестирование.

9.2.3. Неуказанные версии API и зависимостей

В документации не указано, какие версии библиотек, плагинов, движка и SDK использовались. В разделе "Установка пакета" упоминается "последняя версия Godot Engine", но это недопустимо в официальной документации — требуется фиксировать точную версию, например: "Godot Engine 4.4.1 (build 220909.1)". Это критично для воспроизводимости и совместимости. То же относится к аддонам: какие из них активированы, какова их версия, где взять? Рекомендуется создать раздел "Используемые зависимости и лицензии", со списком всех подключённых компонентов.

9.3 Подведение итогов

9.3.1 Общая оценка

Отчет другой команды по проекту VOXGORE носит фрагментарный и неполный характер. Документация не содержит технического задания, отсутствует описание требований, целей, ограничений и проектных решений. Архитектура описана поверхностно, без схем и взаимосвязей компонентов. Руководство пользователя уклоняется от конкретики и не охватывает исключительных сценариев. Формат тестирования — упрощённый, без чёткой структуры, воспроизводимости и привязки к исходному коду.

Общий стиль подачи информации не соответствует требованиям к инженерной документации. Отчет больше напоминает внутренние заметки команды, чем полноценный документ, готовый к передаче заказчику или интеграции в производственный процесс. Требуется глубокая переработка с включением всех обязательных разделов, унификацией формата и повышением точности формулировок.

ГЛАВА 10. ЗАКЛЮЧЕНИЕ

В ходе проделанной работы была проведена всесторонняя оценка как собственного программного продукта, так и документации другой команды. Это позволило глубже понять не только процесс тестирования и верификации программного обеспечения, но и важность полноценного проектного документооборота: от технического задания до финальных сценариев тестирования. Работа показала, что качество документации напрямую влияет на удобство поддержки, масштабируемости и устойчивость проекта при передаче другим разработчикам или внешним проверяющим.

Анализ чужой документации выявил типовые ошибки: отсутствие структурированного ТЗ, неформализованные тест-кейсы, слабая связь между архитектурой и поведением системы. Эти недочёты подчеркнули ценность стандартизированных подходов и строгого следования логике проектирования, которую мы постарались соблюсти в собственной разработке. Практика сравнения двух подходов научила внимательности к деталям, уместности формулировок, а также необходимости делать документацию не только для галочки, но и как рабочий инструмент.

Главный итог — понимание, что грамотное описание системы должно быть не менее приоритетным, чем её кодовая реализация. Именно документация помогает донести замысел, продемонстрировать работоспособность и облегчить сопровождение. Эта работа научила мыслить как разработчик, но действовать как инженер: системно, полно и с ориентацией на дальнейшее использование продукта.